1. (15 marks) Initial value problem

Consider the initial value problem

$$y'(t) = \left(\frac{y(t)}{20}\right)(10 - y(t)), \quad y(0) = \frac{1}{5}.$$

(a) Show that

$$y(t) = \frac{10}{49e^{-\frac{t}{2}} + 1}$$

is an exact closed-form solution to this particular initial value problem (IVP), including the initial condition. You do not need to solve the ODE from scratch; demonstrate that the proposed solution satisfies the ODE and initial condition.

LHS: $y'(t) = -10\left(49e^{-\frac{t}{2}} + 1\right)^{-2}\left(49\left(-\frac{1}{2}\right)e^{-\frac{t}{2}}\right) = 245\frac{e^{-\frac{t}{2}}}{\left(49e^{-\frac{t}{2}}+1\right)^2}$. RHS: $\left(\frac{y(t)}{20}\right)(10 - y(t)) = \frac{\frac{1}{2}}{49e^{-\frac{t}{2}}+1}\left(10 - \frac{10}{49e^{-\frac{t}{2}}+1}\right) = 245\frac{e^{-\frac{t}{2}}}{\left(49e^{-\frac{t}{2}}+1\right)^2}$. $y(0) = \frac{10}{49+1} = \frac{1}{5}$. So the proposed solution satisfies the ODE and initial condition.

(b) Create a single Jupyter notebook for the rest of this question, named `A2Q1.ipynb`. Define a function `Dynamics(t, y)` that implements the dynamics function for the problem above. That is, it should return the derivative `y'(t) = f(t, y)` for this problem. Define a second function `ExactVal(t)` that returns the exact solution at time `t`, as given in part (a).

```
import numpy as np
def Dynamics(t, y):
    return (y/20)*(10-y)
def ExactVal(t):
    return 10/(49*np.exp(-t/2)+1)
```

(c) Define a function `ForwardEuler(t0, tfinal, N, y0)` that takes the initial time `t0`, final time `tfinal`, number of time steps `N`, and initial value `y0`, and implements the Forward Euler method. It should return the approximate solution as a vector `y` of length `N + 1`, such that

$$y_i \approx y(t_i), \quad i = 0, 1, 2, ..., N,$$

where

$$t_i = t_0 + ih, \quad h = \frac{t_{\text{final}} - t_0}{N},$$

and `y(t_i)` denotes the exact solution at time `t = t_i`. Your `ForwardEuler` function should call the `Dynamics` function from part (b) when needed.

```
def ForwardEuler(t0, tfinal, N, y0):
    h = (tfinal-t0)/N
    y = np.zeros(N+1)
    y[0] = y0
    for i in range(N):
        t_i = t0+i*h
        y[i+1] = y[i]+h*Dynamics(t_i, y[i])
    return y
```

(d) Apply `ForwardEuler` to solve the IVP to `tfinal = 20`, first with `N1 = 5` constant-size time steps, then with `N2 = 10`, and finally with `N3 = 20`. Plot the three numerical solutions together on one graph over `0 <= t <= 20`, using the `+` symbol for the first solution, the `o` symbol for the second, and the `*` symbol for the third. On the same graph, plot the exact analytical solution using solid lines. Evaluate the exact solution at the same discrete times as the `N3` solution.

```
import matplotlib.pyplot as plt
t0 = 0
tfinal = 20
N1 = 5
N2 = 10
N3 = 20
y0 = 1/5
for N in [N1, N2, N3]:
    y = ForwardEuler(t0, tfinal, N, y0)
    t = np.linspace(t0, tfinal, N+1)
    plt.plot(t, y, linestyle='None', marker='+' if N==N1 else 'o' if N==N2 else '*', label=f'N={N}')
t_exact = np.linspace(t0, tfinal, N3+1)
```

```
y_exact = ExactVal(t_exact)
plt.plot(t_exact, y_exact, label='Exact', color='black')
plt.xlabel('t')
plt.ylabel('y')
plt.title('Numerical Solutions of the IVP')
plt.legend()
plt.show()
```

The plot should show that the numerical solutions converge to the exact solution as `N` increases.

(e) The explicit midpoint time-stepping rule, a second-order Runge–Kutta scheme, is

$$k_1 = f(t_n, y_n),$$
$$k_2 = f\left(t_n + \frac{h}{2}, y_n + h\frac{k_1}{2}\right),$$
$$y_{n+1} = y_n + hk_2.$$

Define a function `ExplicitMidpoint(t0, tfinal, N, y0)` that implements this method, similarly to `ForwardEuler` in part (c).

```
def ExplicitMidpoint(t0, tfinal, N, y0):
    h = (tfinal-t0)/N
    y = np.zeros(N+1)
    y[0] = y0
    for i in range(N):
        t_i = t0+i*h
        k1 = Dynamics(t_i, y[i])
        k2 = Dynamics(t_i + h/2, y[i] + h*k1/2)
        y[i+1] = y[i] + h*k2
    return y
```

(f) Apply both `ForwardEuler` and `ExplicitMidpoint` to solve the IVP to `tfinal = 20` with `N = 20` constant-size time steps. Plot both numerical solutions together on one graph over `0 <= t <= 20`, using the `+` symbol for Forward Euler and the `o` symbol for midpoint. Also plot the analytical solution using solid lines. Evaluate the exact solution at the same discrete times as the numerical solutions.

```
N = 20
y_fe = ForwardEuler(t0, tfinal, N, y0)
y_mp = ExplicitMidpoint(t0, tfinal, N, y0)
t = np.linspace(t0, tfinal, N+1)
plt.plot(t, y_fe, linestyle='None', marker='+', label='Forward Euler')
plt.plot(t, y_mp, linestyle='None', marker='o', label='Explicit Midpoint')
plt.plot(t, ExactVal(t), label='Exact', color='black')
plt.xlabel('t')
plt.ylabel('y')
plt.title('Numerical Solutions of the IVP')
plt.legend()
plt.show()
```

The plot should show that the midpoint method is more accurate than Forward Euler for the same number of time steps.

(g) Calculate numerical evidence illustrating the order of the Forward Euler method. Let `err(h)` denote the absolute global error at `tfinal` after computing with step size `h`; this is not the local truncation error for a single step. For a method of order `p`,

$$\mathrm{err}(h) \approx Ch^p$$

for some constant `C`. When the step size is halved, we therefore expect

$$\frac{\mathrm{err}\left(\frac{h}{2}\right)}{\mathrm{err}(h)} \approx \frac{1}{2^p}.$$

Thus, the order `p` can be illustrated by calculating these ratios for decreasing `h`.

For Forward Euler, calculate the solution at `tfinal = 10` (different from the final time in earlier parts), halving the step size repeatedly and restarting the time-stepping process for each step size. Use `N = 5 * 2^i` time steps for `i = 0, 1, ..., 9`. Calculate the error at the final time for each step size and the ratio `err(h/2) / err(h)` for successive choices. Print the

results as rows in a simple text table; `print` and tab characters are sufficient. The columns should be the index `i`, the error, and the error ratio relative to the preceding row. The first row has no error ratio. Based on the experimental evidence, what is the order `p` of Forward Euler?

```
print("i\terr(h)\terr(h/2)/err(h)")
tfinal = 10
prev_err = None
for i in range(10):
    N = 5 * 2**i
    y_fe = ForwardEuler(t0, tfinal, N, y0)
    err = abs(y_fe[-1] - ExactVal(tfinal))
    if prev_err is None:
        print(f"{i}\t{err:.2e}\t-")
    else:
        ratio = err / prev_err
        print(f"{i}\t{err:.2e}\t{ratio:.2f}")
    prev_err = err
```

Based on the printed table, the error ratio should approach 0.5 as `i` increases, indicating that the order `p` of Forward Euler is approximately 1.

(h) Repeat part (g) using `ExplicitMidpoint`. What is the order of the explicit midpoint method?

```
print("i\terr(h)\terr(h/2)/err(h)")
prev_err = None
for i in range(10):
    N = 5 * 2**i
    y_mp = ExplicitMidpoint(t0, tfinal, N, y0)
    err = abs(y_mp[-1] - ExactVal(tfinal))
    if prev_err is None:
        print(f"{i}\t{err:.2e}\t-")
    else:
        ratio = err / prev_err
        print(f"{i}\t{err:.2e}\t{ratio:.2f}")
    prev_err = err
```

Based on the printed table, the error ratio should approach 0.25 as `i` increases, indicating that the order `p` of the explicit midpoint method is approximately 2.

2. (8 marks) Improved Euler and Backward Euler

Consider the initial value problem

$$x'(t) = -2y(t) + \frac{t}{2} + 2,$$
$$y'(t) = -1 + 3x(t) - y(t),$$

with initial conditions `x(2) = 3` and `y(2) = 0`. Showing your work, with the aid of a calculator, carry out the following calculations:

(a) Perform two time steps of Improved Euler on this IVP using a time step of `h = 1/2`.
step 1 :

$t_0 = 2, x_0 = 3, y_0 = 0$

$k_1^x = -2(0) + \frac{2}{2} + 2 = 3, k_1^y = -1 + 3(3) - 0 = 8$

$x_1^* = 3 + 0.5(3) = 4.5, y_1^* = 0 + 0.5(8) = 4$

$k_2^x = -2(4) + \frac{2.5}{2} + 2 = -4.75, k_2^y = -1 + 3(4.5) - 4 = 8.5$

$x_1 = 3 + \frac{0.5}{2}(3 - 4.75) = 2.5625, y_1 = 0 + \frac{0.5}{2}(8 + 8.5) = 4.125$
step 2 :

$k_1^x = -2(4.125) + \frac{2.5}{2} + 2 = -5.0, k_1^y = -1 + 3(2.5625) - 4.125 = 2.5625$

$x_2^* = 2.5625 + 0.5(-5.0) = 0.0625, y_2^* = 4.125 + 0.5(2.5625) = 5.40625$

$k_2^x = -2(5.40625) + \frac{3}{2} + 2 = -7.3125, k_2^y = -1 + 3(0.0625) - 5.40625 = -6.21875$

$x_2 = 2.5625 + \frac{0.5}{2}(-5.0 - 7.3125) = -0.515625, y_2 = 4.125 + \frac{0.5}{2}(2.5625 - 6.21875) = 3.2109375$

(b) Perform two time steps of Backward Euler (Implicit Euler) on this IVP using a time step of `h = 1/2`.
step 1 :

$x_1 = x_0 + h\left(-2y_1 + \frac{t_1}{2} + 2\right) \Rightarrow x_1 = 3 + 0.5\left(-2y_1 + \frac{2.5}{2} + 2\right) = 4.625 - y_1$

$y_1 = y_0 + h(-1 + 3x_1 - y_1) \Rightarrow y_1 = 0 + 0.5(-1 + 3x_1 - y_1) = -0.5 + 1.5x_1 - 0.5y_1$

Sub x_1 into the equation for y_1 :

$y_1 = -0.5 + 1.5(4.625 - y_1) - 0.5y_1 \Rightarrow y_1 \approx 2.145833$

$x_1 = 4.625 - 2.145833 = 2.479167$

step 2 :

$x_2 = x_1 + 0.5\left(-2y_2 + \frac{3}{2} + 2\right) = 4.229167 - y_2$

$y_2 = y_1 + 0.5(-1 + 3x_2 - y_2) = 2.145833 + 0.5(-1 + 3x_2 - y_2) = 1.645833 + 1.5x_2 - 0.5y_2$

Sub x_2 into the equation for y_2 :

$y_2 = 1.645833 + 1.5(4.229167 - y_2) - 0.5y_2 \Rightarrow y_2 \approx 2.663194$

$x_2 = 4.229167 - 2.663194 \approx 1.565972$

3. (4 marks) Convert a third-order ODE to a first-order system

Consider the third-order differential equation

$$y'''(t) - 7 - 2y''(t) = \cos(t)y'(t),$$

with initial conditions at `t_0 = 3`:

$$y(3) = 3, \quad y'(3) = -1, \quad y''(3) = 2.$$

Convert this IVP into a system of first-order ordinary differential equations, including appropriate initial conditions for the new variables.

Let $y_1 = y$, $y_2 = y'$, and $y_3 = y''$. Then the system of first-order ODEs is:

$$\begin{aligned} y_{1'} &= y_2, \\ y_{2'} &= y_3, \\ y_{3'} &= 7 + 2y_3 + \cos(t)y_2, \end{aligned}$$

with initial conditions $y_1(3) = 3$, $y_2(3) = -1$, and $y_3(3) = 2$.

4. (6 marks) Stability of a third-order Runge–Kutta method

Consider the following third-order Runge–Kutta (RK3) method:

$$k_1 = hf(t_n, y_n),$$
$$k_2 = hf\left(t_n + \frac{h}{2}, y_n + \frac{k_1}{2}\right),$$
$$k_3 = hf(t_n + h, y_n + 2k_2 - k_1),$$
$$y_{n+1} = y_n + \frac{k_1 + 4k_2 + k_3}{6}.$$

Analyze the stability of this explicit time-stepping method on the test equation

$$y'(t) = -\lambda y(t), \quad \lambda > 0,$$

considering only real `lambda` (not complex values). Determine the conditions on `h` under which the method is expected to be stable. Begin by applying the RK3 scheme to the test equation. As in the stability analyses of Forward Euler and second-order Runge–Kutta, arrive at a condition of the form $|g(x)| < 1$ where `g` is a polynomial in $x = \lambda h$.

Generate a plot of $|g(x)| - 1$ and use it to determine approximate zeros by visually identifying where the graph crosses zero, to at least one decimal place. Use these zeros to determine an approximate stability condition for `h` of the form `???` `< h <` `???`, replacing the placeholders with expressions. Include the plot in your answer and give all coefficients to at least one decimal place.

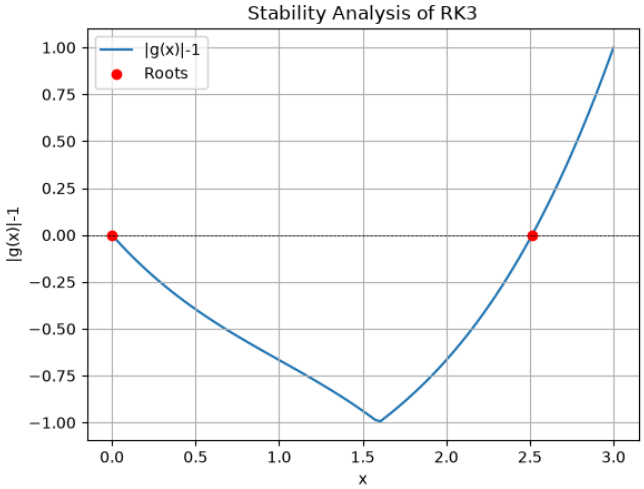
$k_1 = h(-\lambda y_n) = -xy_n$

$k_2 = h\left(-\lambda\left(y_n + \frac{k_1}{2}\right)\right) = -x\left(y_n + \frac{k_1}{2}\right) = -xy_n + \frac{x^2}{2}y_n$

$k_3 = h(-\lambda(y_n + 2k_2 - k_1)) = -x\left(y_n + 2\left(-xy_n + \frac{x^2}{2}y_n\right) + xy_n\right) = -xy_n + x^2y_n - x^3y_n$

$y_{n+1} = y_n + \frac{1}{6}\left(-xy_n + 4\left(-xy_n + \frac{x^2}{2}y_n\right) + (-xy_n + x^2y_n - x^3y_n)\right) = \left(1 - x + \frac{1}{2}x^2 - \frac{1}{6}x^3\right)y_n$

so $\quad g(x) = 1.0 - 1.0x + 0.5x^2 - 0.1667x^3, \quad$ stable iff $\quad |g(x)| < 1$



The graph of $|g(x)| - 1$ crosses zero at $x = 0$ and at $x \approx 2.5$: solving $g(x) = -1$ exactly gives $x \approx 2.5127$, and $|g(x)| - 1 < 0$ for $0 < x < 2.5127$ while it is positive for $x > 2.5127$ (there is no other positive crossing, since $g(x) = 1$ only at $x = 0$). Stability therefore requires $0 < \lambda h < 2.51$, i.e.

$$0 < h < \frac{2.51}{\lambda}.$$

5. (6 marks) A two-step time-stepping method

Consider solving a differential equation `y'(t) = f(t, y(t))` using

$$y_{n+1} = y_n + \frac{h}{4}f(t_{n-1}, y_{n-1}) + \frac{3h}{4}f(t_{n+1}, y_{n+1}).$$

The time step is constant:

$$h = t_{n+1} - t_n = t_n - t_{n-1}.$$

(a) State whether this is an explicit or implicit method.
Implicit, since y_{n+1} appears on both sides of the equation.
(b) State whether this is a single-step or multi-step method.
Multi-step method (two-step): computing y_{n+1} needs f evaluated at two earlier time levels, t_n and t_{n-1}, so a separate starting procedure is required and the method is not single-step. (It is also implicit, since y_{n+1} appears on both sides, but that is a separate property.)
(c) Determine the local truncation error in the form

$$\mathrm{LTE} = Ch^{\alpha} y^{\beta}(t_n) + O(h^{\alpha+1}).$$

Determine the values of `alpha`, `beta`, and `C`. The notation `y^(beta)` represents the `beta`th derivative of `y(t)`.
$\tau = y(t_n + h) - y(t_n) - \frac{h}{4} y'(t_n - h) - \frac{3h}{4} y'(t_n + h)$

let $y = y(t_n), y' = y'(t_n)$, etc,

$y(t_n + h) = y + hy' + \frac{h^2}{2} y'' + \frac{h^3}{6} y''' + O(h^4)$

$y'(t_n + h) = y' + hy'' + \frac{h^2}{2} y''' + O(h^4)$

$y'(t_n - h) = y' - hy'' + \frac{h^2}{2} y''' + O(h^4)$ so, $\tau = hy' + \frac{h^2}{2} y'' + \frac{h^3}{6} y''' - \frac{h}{4}\left(y' - hy'' + \frac{h^2}{2} y'''\right) - \frac{3h}{4}\left(y' + hy'' + \frac{h^2}{2} y'''\right) + O(h^4)$
$= h\left(1 - \frac{1}{4} - \frac{3}{4}\right) y' + h^2\left(\frac{1}{2} + \frac{1}{4} - \frac{3}{4}\right) y'' + h^3\left(\frac{1}{6} - \frac{1}{8} - \frac{3}{8}\right) y''' + O(h^4)$
$= -\frac{1}{3} h^3 y'''(t_n) + O(h^4)$ So, the value of alpha, beta, and C is 3, 3, and $-\frac{1}{3}$ respectively.

6. (12 marks) Simulating a basic mass-spring system

Introduction
Use ordinary differential equations to approximate the physics of two projectiles connected by a massless, stretchy, straight spring. The projectiles are thrown in the air and are subject to gravity. Solve this problem approximately using SciPy's numerical tools for ODEs rather than implementing a time-stepper such as Forward Euler or midpoint. Start from the provided `MassSpring.ipynb` notebook and complete the components marked with `TODO` to generate the trajectories of the two projectiles.
In particular, implement the dynamics function for a first-order system of ODEs describing the motion of the projectiles, specify the vector of initial conditions, and modify the code to halt when either projectile first impacts the ground.

Mathematical model
The fixed Cartesian coordinate system has a horizontal `x`-axis and a vertical `y`-axis. The projectiles move both vertically and horizontally, with positions

$$(x_1(t), y_1(t)) \quad \text{and} \quad (x_2(t), y_2(t)),$$

and horizontal and vertical velocities $(u_1(t), v_1(t))$ and $(u_2(t), v_2(t))$, respectively. The forces acting on the projectiles are gravity and a simple spring. Their masses are `m_1` and `m_2`.
Velocity is the first derivative of position: $u_1(t) = x_1'(t)$ and $v_1(t) = y_1'(t)$ are the horizontal and vertical velocities of projectile
1. Likewise, $u_1'(t)$ and $v_1'(t)$ are its horizontal and vertical
accelerations. The same relationships apply to projectile 2. Gravity contributes an acceleration `g` to the vertical component of each projectile's acceleration.
The spring exerts equal and opposite forces on the projectiles. The force is proportional to the spring strength `k`, the difference between its current length l_{cur} and its unstretched rest length l_0, and the direction vector `d` along the spring. The vector notation indicates vectors of length
2. The spring pulls or pushes its endpoints to return to its rest length.
The forces are

$$F_{12} = -k(l_{\text{cur}} - l_0)d,$$
$$F_{21} = -F_{12}.$$

Here

$$\Delta = (x_1(t) - x_2(t), y_1(t) - y_2(t)),$$
$$l_{\text{cur}} = \|\Delta\|,$$
$$d = \frac{\Delta}{l_{\text{cur}}}.$$

Thus, `Delta` points from one projectile to the other, l_{cur} is the current spring length, and `d` is a unit vector along the spring. F_{12} is the force on projectile 1 due to the spring, and F_{21} is the force on projectile 2 due to the same spring. These equations are Hooke's law.

The motion of the two projectiles is described by the following combined system of eight ordinary differential equations:

$$x_1'(t) = u_1(t), \quad y_1'(t) = v_1(t),$$
$$x_2'(t) = u_2(t), \quad y_2'(t) = v_2(t),$$
$$u_1'(t) = \frac{F_{x,12}}{m_1}, \quad v_1'(t) = g + \frac{F_{y,12}}{m_1},$$
$$u_2'(t) = \frac{F_{x,21}}{m_2}, \quad v_2'(t) = g + \frac{F_{y,21}}{m_2}.$$

The `x` and `y` subscripts on the force vectors indicate the component used. Since the state of each projectile affects the other, solve the equations as one combined system.

Problem parameters

Use gravitational acceleration `g = -9.81`, projectile masses $m_1 = 1$ and $m_2 = 5$, spring rest length $l_0 = 4$, and spring strength `k = 3`. The starting time is $t_0 = 0$. The end time is $t_{\text{final}} = 30$, unless a projectile hits the ground first.

Initial conditions

The initial position of projectile 1 is $(x_1, y_1) = (0, 25)$, and the initial position of projectile 2 is $(x_2, y_2) = (5, 25)$. The launch velocity of projectile 1 is $(u_1, v_1) = (2, 20)$. The launch velocity of projectile 2 is $(u_2, v_2) = (-21, 30)$.

Instructions

The provided `MassSpring.ipynb` notebook contains most of the code needed to simulate this ODE system. Work out the dynamics function and initial conditions from the model, then fill in the corresponding `TODO` locations.

The code is nearly set up to halt at the instant either projectile hits ground level `y = 0`, using the event functionality of SciPy's ODE solver. Fill in the ground-event function so that it returns the lower of the two projectiles' current `y` values. SciPy's `solve_ivp` function accepts the dynamics function, simulation time span, initial conditions, and additional parameters. It returns the discrete time instants at which the solution is calculated and the state at those times. For more information, consult the `solve_ivp` documentation and the `ode_suite_demos.ipynb` example posted on LEARN under Contents → Resources → Code Samples:

SciPy `solve_ivp` documentation

```
# Dynamics function for the ODE
def dynamics_function(t, z):
    # The dynamics function accepts time t and vector z as input, and
    # returns the resulting vector of derivative values given
    # by z' (i.e., the vector dzdt)

    #the entries of the incoming z vector will contain the following data
    #z[0] = x-position of projectile 1, x1(t)
    #z[1] = y-position of projectile 1, y1(t)
    #z[2] = x-position of projectile 2, x2(t)
    #z[3] = y-position of projectile 2, y2(t)
    #z[4] = x-velocity of projectile 1, u1(t)
    #z[5] = y-velocity of projectile 1, v1(t)
    #z[6] = x-velocity of projectile 2, u2(t)
    #z[7] = y-velocity of projectile 2, v2(t)
    #this is essentially the current state of the ODE system
```

```
    #create the empty z' vector
    dzdt = np.zeros(8);

    #*************
    #TODO HERE: Fill 'dzdt' with a vector of the necessary derivative data according to the problem's
dynamics function,
    #           consistent with the ordering of variables listed above
    m_1, m_2 = 1, 5
    l_0, k = 4, 3
    g = -9.81

    delta = np.array([z[0] - z[2], z[1] - z[3]])
    l_cur = np.linalg.norm(delta)
    d = delta / l_cur

    f_12 = -k * (l_cur - l_0) * d
    f_21 = -f_12

    dzdt = [
        z[4], z[5], z[6], z[7],
        f_12[0] / m_1,
        g + f_12[1] / m_1,
        f_21[0] / m_2,
        g + f_21[1] / m_2,
    ]

    #*************

    # return the vector for SciPy's solver to use.
    return dzdt;

# We'll also define an "event function" to terminate the simulation
# when one of the two projectiles first hits the ground (y=0).
# The IVP solver will check this condition behind the scenes for us, based on the function below.
def ground_event(t,z):
    #*************
    #TODO HERE: set the variable 'miny' to the lower of the two y-values of the two objects,
    #extracted from the current state vector z
    miny = min(z[1],z[3])

    #*************
    return miny

#TODO HERE: Fill the vector z0 below with the necessary initial state data.
#           Refer to the comment in the dynamics_function for the ordering of variables.
z0 = [0,25,5,25,2,20,-21,30]
```

Simulated trajectories

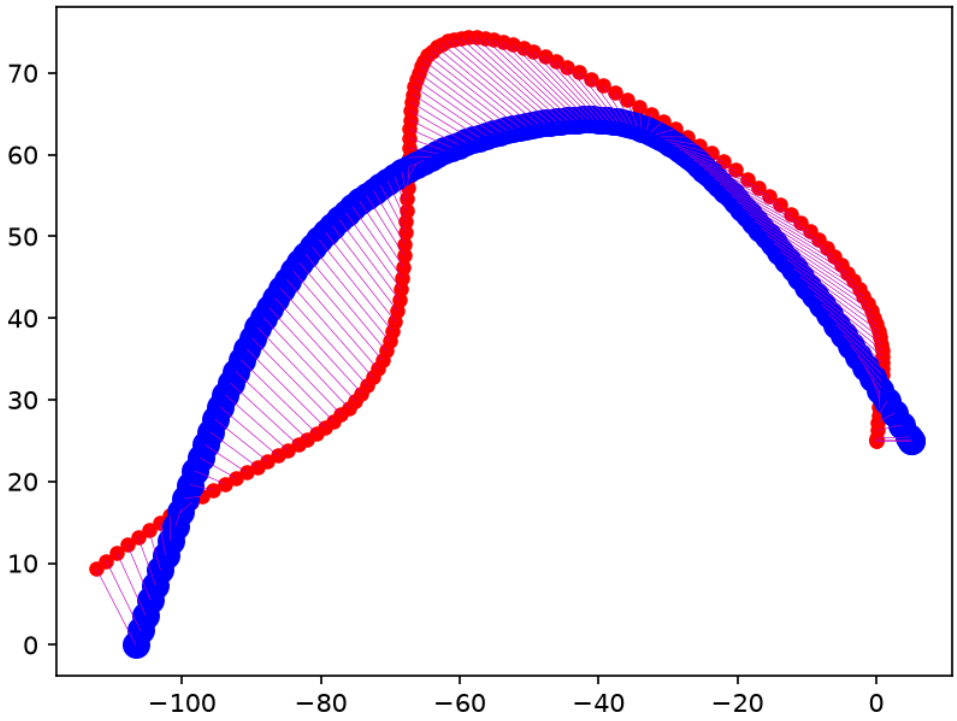


Figure 2: Trajectories produced by the completed `MassSpring.ipynb`, with the connecting spring drawn at every time step (red: lighter projectile 1, blue: heavier projectile 2). The ground event stops the integration at $t \approx 6.51$ s, when projectile 2 reaches $y = 0$ first, while projectile 1 is still at $y \approx 9.3$ m.

Both projectiles are launched upwards, arc over the peak (about 74 m for projectile 1 and 64 m for projectile 2), and the spring — whose length varies between roughly 2 m and 17 m — keeps pulling them together while they drift far to the left, driven by the launch velocity $u_2 = -21$ of the heavier projectile.

Required submission

Your answer to this question should consist of:

1. A modified `MassSpring.ipynb` notebook with the dynamics function, initial conditions, and ground-event function filled in at the appropriate `TODO` locations.
2. A static plot of the trajectories produced by running the completed notebook. You should not need to modify the visualization or plotting code to produce the figure.

Optional: The notebook contains a final code block that can generate and play an animation of the projectiles and connecting spring, provided your Jupyter/Python environment supports it. Depending on your environment, you may need to make further changes or debug this block. Running it is not required, but observing the animation can be interesting and may provide intuition about the projectiles' motion or help with debugging.